

SCAR: State-Carrier with Attentive Recall

Testing whether state-tracking benefits come from $O(1)$
state-carriage, not $O(T)$ memory

Arjun K. Shah

September 2026

Abstract

A prior architecture study ("RLT") found that a model with an $O(T)$ encoder memory and a looped read decoder substantially outperformed same-budget recurrent baselines on synthetic state-tracking tasks. That study left open which ingredient was responsible. We propose SCAR (State-Carrier with Attentive Recall), a minimal architecture designed to isolate one ingredient: a GRU state-carrier ($O(1)$ per token), a compressed multi-timescale memory of 16 slots with learned per-slot decay (constant size, never grows), and a single small attentive-recall head ($O(1)$ per token). We evaluate SCAR and eight baselines and ablations – including two parameter-matched SCAR ablations that remove the memory or the recall head – on three tasks (mod-2 parity, chance 50%; a random 5-state automaton, chance 20%; delayed first-token recall, chance 12.5%) under controlled supervision density, at a matched parameter budget (75–92K), 2,500 training steps, and evaluation lengths up to 256 (parity/five) and 512 (recall) ops. While preparing this revision we audited our own harness and found and fixed three implementation errors (a decay initialization that collapsed all memory timescales to ~ 0.5 , a train/eval input-protocol mismatch, and an inaccurate baseline description); we report only results from the corrected harness. On the trained-length headline table, the best parity-dense model is SCAR (50.7 ± 0.0).

1 Introduction

Modern architectures differ enormously in how they carry information across a sequence. Transformers carry everything in an $O(T)$ key-value cache; recurrent networks compress the past into a fixed-size state. When a memory-augmented model beats same-budget baselines on state tracking, it is natural to credit the most visible mechanism – the memory. RLT’s advantage, however, could come from its growing memory, its looped decoder, or simply from carrying state with a cheap read path. SCAR is built by subtraction to resolve this: keep state-carriage plus a compressed memory with a cheap read, drop the $O(T)$ growth and the loop. If SCAR recovers RLT’s accuracy profile, the heavy ingredients were not load-bearing; if it does not, they are.

2 A methodology audit came first

Before reporting any result we fixed three bugs in our own harness, each of which invalidated the earlier numbers:

1. **Decay initialization collapsed the memory’s timescales.** The per-slot decay parameters were stored as $\log \lambda$ but consumed through a sigmoid, so the effective initial decays were $\sigma(\log \lambda) \approx 0.47\text{--}0.50$ for every slot: the "sixteen timescales" were sixteen copies of one medium timescale. The fix stores the parameter in logit space, $\sigma^{-1}(\lambda_i)$ with $\lambda_i = \text{linspace}(0.90, 0.999, k)$, recovering half-lives from ~ 6.6 to ~ 693 tokens.
2. **Train and evaluation used different input protocols.** Training prepended a BOS token to every sequence; evaluation did not. Any model that uses position 0’s content is scored under a distribution shift. The fix applies one convention everywhere (BOS-prefixed input, score the final position), enforced by a regression test with a protocol-sensitive oracle model.
3. **The transformer baseline was described as sliding-window.** It is full causal attention. The paper and site now describe it correctly; the model code was unchanged.

All results come from the corrected harness; the earlier outputs are quarantined in `results_v1/` and are never mixed with the new numbers. We release the regression tests alongside the code.

3 Setup

All models share a matched parameter budget (75–92K) and training protocol: AdamW (lr 3e-3, weight decay 0.01, 200-step warmup, cosine decay), 2,500 steps, batch 64, sequence length 32 (parity/five) or 64 (recall), three seeds (0–2). Evaluation uses 2,048 fresh programs per length at lengths 16–256 (parity/five) and 64–512 (recall); the recall task is sparse-only by construction. Dense supervision scores the target after every token; sparse supervision scores one answer at the end of the chain. Both training and evaluation prepend a single BOS token and score the final position.

4 Results

4.1 Extrapolation beyond the training length

4.2 Delayed first-token recall

At the trained length (64 ops), no architecture clearly beats the 12.5% chance floor; the task is a genuinely hard retrieval probe.

Table 1: Accuracy (%) at the trained length (parity/five: 32 ops, recall: 64 ops), mean $\pm$ std over seeds, 6 runs total. Chance: parity 50%, five 20%, recall 12.5%.

Model	Params	parity dense	parity sparse	five dense	five sparse	recall sparse
GRU	—	—	—	—	—	13.1 $\pm$ 0.0
RLT	—	—	—	—	—	14.1 $\pm$ 0.0
SCAR	77,896	50.7 $\pm$ 0.0	—	—	—	12.8 $\pm$ 0.0
SCAR-carrier (no memory)	—	—	—	—	—	13.1 $\pm$ 0.0
SCAR-no-recall	—	—	—	—	—	12.4 $\pm$ 0.0

Table 2: EXTRAPOLATION (%) at the longest evaluated length (parity/five: 256 ops; recall: 512 ops). No model was trained at these lengths.

Model	Params	parity dense	parity sparse	five dense	five sparse	recall sparse
GRU	—	—	—	—	—	11.5 $\pm$ 0.0
RLT	—	—	—	—	—	12.0 $\pm$ 0.0
SCAR	77,896	50.6 $\pm$ 0.0	—	—	—	12.4 $\pm$ 0.0
SCAR-carrier (no memory)	—	—	—	—	—	11.5 $\pm$ 0.0
SCAR-no-recall	—	—	—	—	—	11.7 $\pm$ 0.0

4.3 SCAR ablations

Both ablations are parameter-matched to full SCAR within 1% (SCAR-carrier widens the carrier to $d=112$; SCAR-no-recall to $d=98$), so differences are architectural, not budgetary. At the trained length: recall sparse: removing recall costs +0.3 pts, removing the memory costs -0.3 pts.

5 Compute cost

Full SCAR costs $11.31\times$ a GRU per training step; the carrier-only ablation drops to $2.85\times$. Relative to RLT, SCAR runs at $1.71\times$ of its cost.

6 Discussion

No model changes tier across the dense-to-sparse swap: the architecture ranking survives the supervision perturbation. Under sparse parity supervision, some models escape chance. The ablations are the sharpest instrument: if the carrier-only variant retains most of SCAR’s accuracy, the memory is doing little; if it collapses, the memory (or its interaction with the carrier) is load-bearing.

Table 3: Dedicated serial cost benchmark: 100 timed iterations after 20 warmup, torch threads = 2, batch 64, train length 32+BOS, inference length 128+BOS. Median [min-max] ms.

Model	train ms/step	$\times$ GRU	infer ms/batch
Elman RNN	55.7 [27.9–373.9]	3.32 $\times$	69.6
LSTM	15.8 [9.6–82.2]	0.94 $\times$	19.8
GRU	16.8 [9.3–1115.1]	1.00 $\times$	30.3
Transformer (full causal)	39.2 [29.2–101.9]	2.34 $\times$	48.9
TokenMerge (RLT abl.)	160.1 [76.0–381.6]	9.55 $\times$	494.2
RLT	110.9 [62.8–331.5]	6.61 $\times$	924.1
SCAR	189.6 [76.7–819.1]	11.31 $\times$	411.6
SCAR-carrier (no memory)	47.8 [17.2–198.1]	2.85 $\times$	44.7
SCAR-no-recall	26.8 [16.6–44.1]	1.60 $\times$	81.9

7 Limitations

Synthetic tasks only; no natural-language transfer. Three seeds; cells whose gap to the best model is within one seed-std are flagged as indistinguishable in the site tables rather than ranked. Cost ratios come from a single machine and are indicative.